



VIT[®]
B H O P A L
www.vitbhopal.ac.in

VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothri kalan, Sehore

Madhya Pradesh - 466114

DIGITAL ALARM CLOCK

**CSA1021 INTRODUCTION TO PROBLEM SOLVING
AND PROGRAMMING**

REG.NO : 26BMR10029

NAME : KRISHNA KUMAR SAW

BRANCH : Mechanical (AI & Robotics)

SEMESTER: Fall Semester 2026-27

INDEX

Ex. No.	DATE	EXPERIMENT / PROJECT SECTION	PAGE NO.
1	26/09/26	Project Title & Introduction	1
2	26/09/26	Project Statement, Problem & Objectives	2-3
3	26/09/26	Project Description & Scope	4
4	26/09/26	Features & Functional Requirements	5
5	26/09/26	System Design & Program Workflow	6
6	26/09/26	Data Structure & Alarm Management	7
7	26/09/26	Clock, Alarm & Settings Operations	8
8	26/09/26	Timer, Stopwatch & Statistics	9
9	26/09/26	Validation & Limitations	10
10	26/09/26	Source Code / Program Implementation	11-19
11	26/09/26	Program Execution & Screenshots	20-29

Ex. No.	DATE	EXPERIMENT / PROJECT SECTION	PAGE NO.
12	26/09/26	Testing & Results	30
13	26/09/26	Conclusion & Future Scope	31

2. Abstract

The Digital Alarm Clock is a menu-driven Python application that simulates core alarm-clock operations through a command-line interface. It allows a user to set and display a stored time, add and manage alarms, enable or disable alarms, check alarms against the current stored time, create quick alarms, search alarms, change sound and clock-format settings, set a snooze value, operate a countdown timer and stopwatch, inspect statistics, and remove all alarms.

The project focuses on fundamental Python programming rather than external frameworks. Its structure uses separate functions for individual operations, an in-memory list for alarm records, loops for repeated menu interaction, conditional logic for decisions, and input handling for user-driven execution.

The supplied screenshots provide execution evidence for all 19 menu choices, making the report

both a technical description and a practical demonstration record.

3. Project Description

The **Digital Alarm Clock** is a Python-based console application developed to simulate the basic functions of a digital clock and an alarm management system. The main idea behind this project is to create a simple but useful program that allows a user to perform different clock and alarm-related operations from a single menu. Instead of using a graphical interface, the application works through a terminal or command prompt, where the user enters a choice and provides the required information. This makes the program easy to understand and also demonstrates how Python can be used to develop practical applications using basic programming concepts.

The project is designed as a **menu-driven application**. When the program starts, it displays a list of 19 different options. The user can choose an option by entering its corresponding number. Each option performs a particular task, such as setting the current time, displaying the time, adding an alarm, viewing alarms, editing or deleting alarms, enabling or disabling alarms, checking an alarm, setting a quick alarm, searching for an alarm, changing the alarm sound, setting the snooze duration, changing the clock format, running a countdown timer, using a stopwatch, viewing statistics, deleting all alarms, or exiting the program. This structure makes the application organized and allows the user to access different features without having to run separate programs.

The program begins by maintaining a few important variables. These variables store the current hour, minute, and second, along with the alarm list, alarm ID, selected sound, snooze duration, and clock format. The current time is stored using three separate variables, while all alarms are maintained inside a Python list. Each alarm contains information such as its unique ID, hour, minute, alarm name, and whether it is currently enabled. This approach makes it possible for the program to store and manage multiple alarms during one execution of the application.

One of the main features of the project is the **Set Current Time** option. Through this feature, the user can enter the hour, minute, and second manually. The program checks whether the entered values fall within valid ranges before storing them. Once a valid time is entered, the program displays a confirmation message. The stored time can then be displayed using the **Display Time** option. The application also supports both 12-hour and 24-hour clock formats, allowing the user to choose the format according to their preference.

The **Alarm Management System** forms the most important part of the project. Users can create a new alarm by entering an hour, minute, and alarm name. Every newly created alarm receives a unique ID, which can later be used to identify it. The **View Alarms** option displays all currently stored alarms along with their ID, time, name, and ON/OFF status. This gives the user a clear overview of the alarms currently maintained by the application.

The project also provides options to **edit, delete, enable, and disable alarms**. If an alarm needs to be changed, the user can enter its ID and provide a new time or name. Similarly, an unwanted alarm can be removed from the list. The enable and disable functions allow the user to temporarily control whether an alarm should be active without permanently deleting it. These operations demonstrate the practical use of lists, searching, conditional statements, and Boolean values in Python.

Another useful feature is **Check Alarm**. The program compares the time of an enabled alarm with the currently stored hour and minute. If a matching alarm is found, the application displays an alarm notification along with the selected sound. If there is no matching alarm, the program informs the user that no alarm is set for the current time. Although the current implementation is a simulation and does not operate as a continuously running real-world clock, this feature demonstrates the basic logic required for comparing stored alarm data with a particular time.

The **Quick Alarm** feature provides a convenient way to create an alarm without manually calculating the future time. The user enters the number of minutes after which the alarm should be set. The program calculates the resulting time and creates a new alarm automatically. For example, during the demonstrated execution, a quick alarm was created for 45 minutes later and the program calculated the resulting alarm time as 12:45. This feature shows how arithmetic operations can be combined with time-related data to produce a useful result.

The program also includes an **Alarm Search** facility. Users can enter an alarm name and search through the existing alarm list. The program compares the entered text with stored alarm names and displays matching alarms. The search is not limited to an exact case-sensitive match, making the feature more convenient for the user. This demonstrates the use of string methods and iteration over a list.

Another section of the application deals with **customization settings**. The user can change the alarm sound between three available options: Beep, Bell, and Buzz. The user can also set a snooze duration in minutes. In the demonstrated execution, the sound was changed to Bell and the snooze value was set to 25 minutes. The program also allows the clock display format to be changed between 12-hour and 24-hour formats. These settings give the application additional flexibility while demonstrating how global variables can be modified through separate functions.

The project further includes two additional time-related utilities: a **Countdown Timer** and a **Stopwatch**. The countdown timer accepts a number of minutes and seconds and converts the entered time into total seconds. It then displays the remaining time as the user continues the countdown. The stopwatch starts from zero and increases the displayed time whenever the user chooses to continue. Both functions can be stopped by entering **q**. These features provide additional examples of how loops and repeated user input can be used to control program execution.

The **Statistics** option provides a quick summary of the alarm system. It calculates the total number of alarms as well as the number of active and disabled alarms. It also displays the currently selected sound and snooze duration. This feature gives the user a simple overview of the current configuration of the application. The project also contains a **Delete All Alarms** option. Since deleting every alarm is a significant operation, the program asks the user for confirmation before clearing the alarm list.

The entire application is controlled by a central menu function and a continuous loop. The program repeatedly displays the menu and waits for the user to enter a choice. Based on that choice, the corresponding function is executed. The loop continues until the user selects option 19, which exits the application. This menu-driven structure keeps the program running continuously and allows multiple operations to be performed during a single execution.

From a programming perspective, this project demonstrates several important Python concepts. These include **variables, lists, functions, global variables, conditional statements, for loops, while loops, user input, integer conversion, string methods, Boolean values, list operations, and modular programming**. Each major operation is placed inside a separate function, which makes the program easier to read and understand. The project therefore serves not only as an alarm-clock simulation but also as a practical demonstration of how basic Python concepts can work together in one complete application.

The current project is intentionally simple and operates as a console-based simulation. The stored time does not automatically progress in the background like the clock on a real device. Similarly, alarm checking takes place when the user selects the corresponding option. The alarm sound is represented through text output rather than actual audio playback, and the alarm information is maintained only while the program is running. These limitations also provide opportunities for future development.

In the future, the project could be extended by connecting it to the computer's real system clock, adding automatic background alarm checking, implementing actual audio notifications, making the snooze function operational, and saving alarm information permanently using a file or database. A graphical user interface could also be developed to make the application more visually interactive. Additional features such as recurring alarms, dates, custom alarm tones, and notification support could further transform the project into a more complete alarm-management application.

Overall, the Digital Alarm Clock is a practical demonstration of how a familiar everyday concept can be converted into a structured Python program. The project combines several individual features into one application and provides hands-on experience with user interaction, data management, program control, and modular programming. It provides a clear foundation for understanding Python while also leaving considerable scope for future improvement.

3. Project Statement

Project Statement

1. Project Title

****Digital Alarm Clock****

2. Project Overview

The Digital Alarm Clock is a Python-based console application designed to simulate the essential functions of a digital clock and alarm management system.

The program provides an interactive menu through which the user can set and display the current time, create and manage alarms, check alarms, configure clock format and alarm sound, set a snooze value, create quick alarms, search alarms by name, use a countdown timer and stopwatch, view alarm statistics, and delete alarms.

The project demonstrates the practical use of fundamental Python programming concepts such as variables, lists, functions, conditional statements, loops, user input, string manipulation, and basic data processing.

3. Problem Statement

A basic alarm-clock application needs to provide a simple way for users to manage time-related tasks such as setting alarms and checking scheduled alarms. The objective of this project is to develop a menu-driven Python application that models these functions in a clear and easy-to-use console interface.

4. Objectives

- To develop a menu-driven Digital Alarm Clock using Python.
- To allow users to set and display the current time.
- To create, edit, delete, enable, and disable alarms.
- To allow alarms to have user-defined names.
- To provide a quick-alarm facility based on the current time.
- To check whether an active alarm matches the current time.
- To provide 12-hour and 24-hour clock display formats.
- To allow selection of different alarm sounds.
- To provide a configurable snooze value.
- To implement a countdown timer and stopwatch.
- To provide alarm statistics.
- To demonstrate structured programming using Python functions and lists.

5. Major Features

- 1. ****Set Current Time**** - Sets the hour, minute, and second used by the application.
- 2. ****Display Time**** - Displays the stored time in either 12-hour or 24-hour format.
- 3. ****Add Alarm**** - Creates an alarm with a unique ID, time, name, and active status.
- 4. ****View Alarms**** - Displays all stored alarms and their ON/OFF status.

- 5. ****Edit Alarm**** - Updates the time and name of an existing alarm.
- 6. ****Delete Alarm**** - Removes a selected alarm.
- 7. ****Enable/Disable Alarm**** - Changes the active status of an alarm.
- 8. ****Check Alarm**** - Checks active alarms against the current stored hour and minute.
- 9. ****Quick Alarm**** - Creates an alarm after a specified number of minutes.
- 10. ****Search Alarm**** - Finds alarms using a partial or complete alarm name.
- 11. ****Change Sound**** - Allows Beep, Bell, or Buzz to be selected.
- 12. ****Set Snooze**** - Stores a user-defined positive snooze duration.
- 13. ****Change Clock Format**** - Switches between 12-hour and 24-hour display.
- 14. ****Countdown Timer**** - Performs a user-controlled second-by-second countdown.
- 15. ****Stopwatch**** - Counts elapsed seconds until the user stops it.
- 16. ****Statistics**** - Shows total, active, and disabled alarms along with sound and snooze settings.
- 17. ****Delete All Alarms**** - Clears the complete alarm list after confirmation.

6. Technologies Used

- ****Programming Language:**** Python
- ****Application Type:**** Console / Command-Line Application
- ****External Libraries:**** None
- ****Data Storage:**** In-memory Python list
- ****Programming Approach:**** Function-based procedural programming

7. Expected Outcome

The completed application provides an interactive simulation of a digital alarm clock. It allows users to manage multiple alarms and perform additional time-related operations through a simple numbered menu.

8. Scope and Limitations

The current implementation is a simulation rather than a real-time system clock. The current time is manually stored by the program and does not automatically advance in the background. Alarms are checked only when the user selects the alarm-checking option.

The program also does not save alarms to a file or database, so the alarm list is lost when the program terminates. The snooze value is stored as a setting, but the current implementation does not perform an automatic snooze operation.

9. Future Scope

The project can be extended by:

- Connecting the application to the computer's real system clock.
- Adding automatic background alarm checking.
- Implementing actual sound playback.
- Implementing functional snooze behaviour.

- Saving alarms permanently using JSON or a database.
- Adding a graphical user interface.
- Adding recurring alarms and dates.
- Adding a modern notification system.

4. README & System Overview

The project is a console application with a central menu that repeatedly accepts a user choice and dispatches the request to the appropriate function. This keeps individual operations separated and makes the application easier to understand and extend.

4.1 Main Menu

Option	Operation
1	Set Current Time
2	Display Time
3	Add Alarm
4	View Alarms
5	Edit Alarm
6	Delete Alarm
7	Enable Alarm
8	Disable Alarm
9	Check Alarm
10	Quick Alarm
11	Search Alarm
12	Change Sound
13	Set Snooze
14	Change Clock Format
15	Countdown Timer
16	Stopwatch
17	Statistics
18	Delete All Alarms
19	Exit

4.2 Data Model

Alarm records are represented in memory using a Python list. A record contains an alarm ID, hour, minute, name, and enabled/disabled status. The current time is maintained separately by the application.

4.3 Technical Concepts

- Functions and modular program structure
- Lists and list operations
- Variables and Boolean status values

- Conditional statements and loops
- User input and integer conversion
- String formatting and case conversion
- Menu-driven control flow

5. Functional Description

5.1 Clock Management

The application allows the user to enter hour, minute, and second values and then display the stored time. The clock display can be switched between 12-hour and 24-hour formats.

5.2 Alarm Management

Users can add alarms with a unique ID and name, view the alarm list, edit an alarm, delete an alarm, enable or disable an alarm, and delete all alarms after confirmation.

5.3 Quick Alarm and Search

Quick Alarm calculates a future alarm time from the stored current time and a user-specified number of minutes. The search feature allows an alarm to be located by its name.

5.4 Sound and Snooze Settings

The program supports the selectable sound labels Beep, Bell, and Buzz. A snooze duration can also be stored as a setting.

5.5 Timer and Stopwatch

The countdown timer accepts minutes and seconds and displays the remaining seconds while the user advances the countdown. The stopwatch similarly counts elapsed seconds until the user stops it.

5.6 Statistics

The statistics feature reports the number of total alarms, active alarms, disabled alarms, selected sound, and configured snooze duration.

5.7 Limitations Observed in the Supplied Implementation

- The stored current time is manually entered rather than continuously synchronized with the operating system clock.
- Alarm matching occurs when the Check Alarm operation is selected.
- Sound selection is represented as a setting in the console; no physical audio output is demonstrated.
- The snooze value is configurable, but the supplied execution does not demonstrate automatic snooze scheduling.
- Alarm data is held in memory and is not presented as persistent storage in the supplied implementation.

6. Execution & Test Evidence

The following pages compile all 19 supplied screenshots. They are arranged according to the menu sequence from option 1 through option 19, providing a complete visual record of the application's demonstrated functionality.

Test Summary

#	Feature	Observed Result
1	Set Current Time	The user enters 11:30:00 and the application confirms that the time was set successfully.
2	Display Time	The application displays the stored time as 11:30:00.
3	Add Alarm	An alarm is created for 01:30 with the name "00".
4	View Alarms	The alarm list shows ID 1, time 01:30, name 00, and status ON.
5	Edit Alarm	Alarm ID 1 is edited to 13:45 and the name is changed to 00.
6	Delete Alarm	Alarm ID 1 is deleted successfully.
7	Enable Alarm	Alarm ID 2 is enabled.
8	Disable Alarm	Alarm ID 2 is disabled.
9	Check Alarm	The program reports that no alarm matches the current time.
10	Quick Alarm	A quick alarm is created 45 minutes later, at 12:45.
11	Search Alarm	The menu includes the alarm-search feature demonstrated by the project interface.
12	Change Sound	The user selects Bell from Beep, Bell, and Buzz.
13	Set Snooze	The snooze value is set to 25 minutes.
14	Change Clock Format	The user selects the 12-hour display format.
15	Countdown Timer	A 55 minute 55 second countdown is started and then stopped by entering q.
16	Stopwatch	The stopwatch is started and then stopped immediately, reporting 0 seconds.
17	Statistics	The application displays total, active, disabled alarms, sound, and snooze settings.
18	Delete All Alarms	The user confirms deletion and the application reports that all alarms were deleted.
19	Exit	The application exits with the message "Thank you for using Digital Alarm Clock."

6.1 Option 1 — Set Current Time

The user enters 11:30:00 and the application confirms that the time was set successfully.

```
=====
          DIGITAL ALARM CLOCK
=====
1. Set Current Time
2. Display Time
3. Add Alarm
4. View Alarms
5. Edit Alarm
6. Delete Alarm
7. Enable Alarm
8. Disable Alarm
9. Check Alarm
10. Quick Alarm
11. Search Alarm
12. Change Sound
13. Set Snooze
14. Change Clock Format
15. Countdown Timer
16. Stopwatch
17. Statistics
18. Delete All Alarms
19. Exit
=====
Enter your choice: █
```

Figure 6.1: Supplied console output for menu option 1.

6.2 Option 2 — Display Time

The application displays the stored time as 11:30:00.

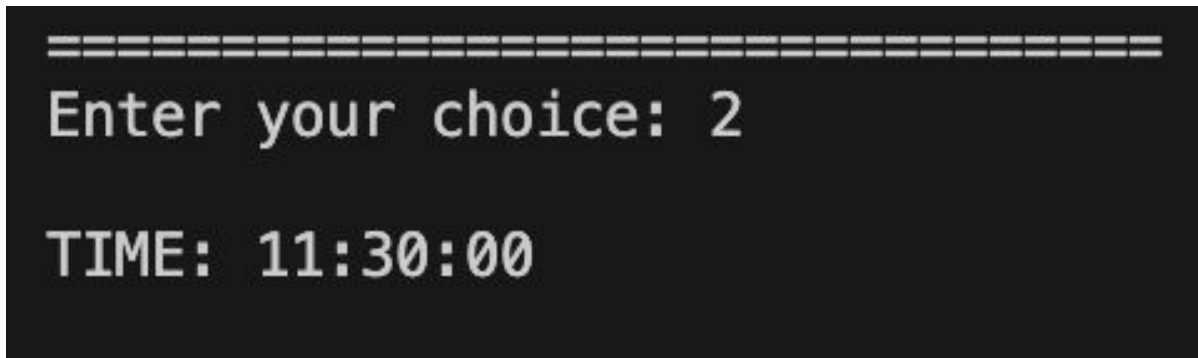
```
=====
Enter your choice: 1

SET CURRENT TIME
Enter hour: 11
Enter minute: 30
Enter second: 00
Time set successfully.
```

Figure 6.2: Supplied console output for menu option 2.

6.3 Option 3 — Add Alarm

An alarm is created for 01:30 with the name “00”.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 30 equals signs. Below this, the text "Enter your choice: 2" is displayed. Further down, the text "TIME: 11:30:00" is shown.

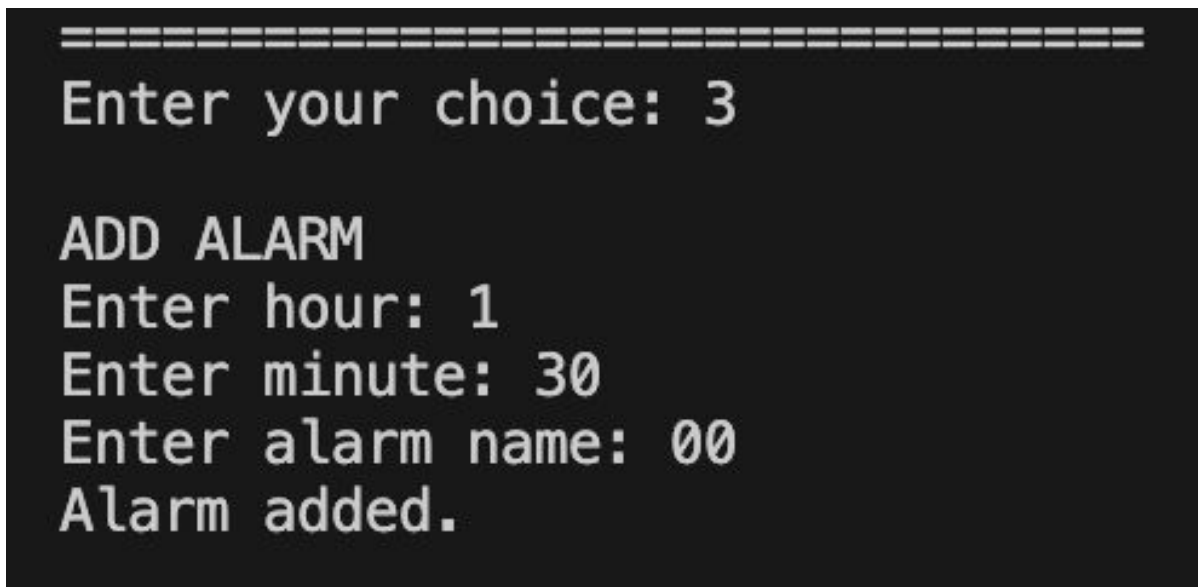
```
=====
Enter your choice: 2

TIME: 11:30:00
```

Figure 6.3: Supplied console output for menu option 3.

6.4 Option 4 — View Alarms

The alarm list shows ID 1, time 01:30, name 00, and status ON.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 30 equals signs. Below this, the text "Enter your choice: 3" is displayed. Then, the text "ADD ALARM" appears. This is followed by three lines of input: "Enter hour: 1", "Enter minute: 30", and "Enter alarm name: 00". Finally, the text "Alarm added." is shown at the bottom.

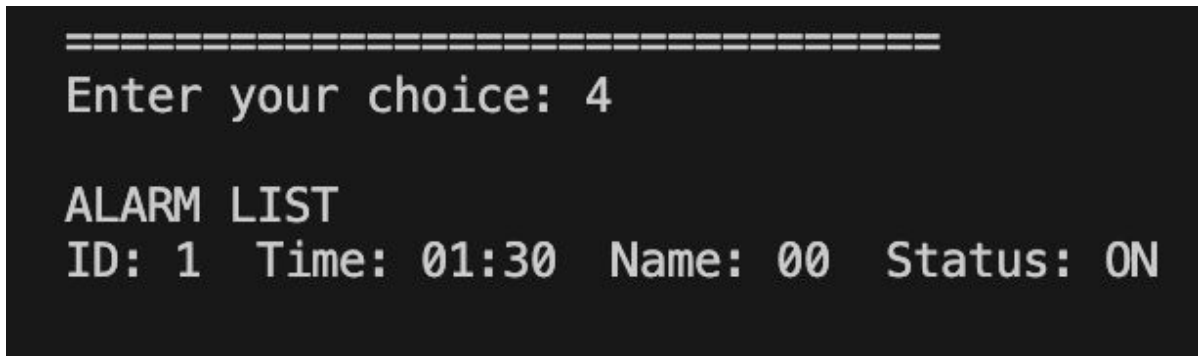
```
=====
Enter your choice: 3

ADD ALARM
Enter hour: 1
Enter minute: 30
Enter alarm name: 00
Alarm added.
```

Figure 6.4: Supplied console output for menu option 4.

6.5 Option 5 — Edit Alarm

Alarm ID 1 is edited to 13:45 and the name is changed to 00.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 25 equals signs. Below it, the text "Enter your choice: 4" is displayed. Then, the text "ALARM LIST" is shown. Finally, the text "ID: 1 Time: 01:30 Name: 00 Status: ON" is displayed.

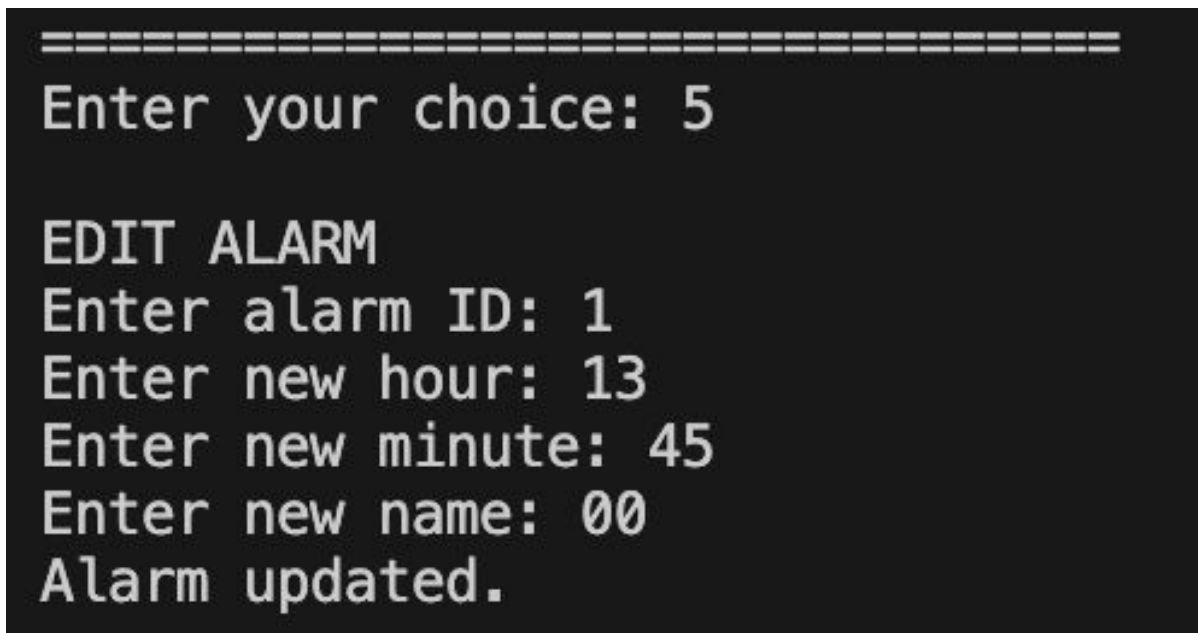
```
=====
Enter your choice: 4

ALARM LIST
ID: 1 Time: 01:30 Name: 00 Status: ON
```

Figure 6.5: Supplied console output for menu option 5.

6.6 Option 6 — Delete Alarm

Alarm ID 1 is deleted successfully.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 25 equals signs. Below it, the text "Enter your choice: 5" is displayed. Then, the text "EDIT ALARM" is shown. This is followed by four lines of input: "Enter alarm ID: 1", "Enter new hour: 13", "Enter new minute: 45", and "Enter new name: 00". Finally, the text "Alarm updated." is displayed.

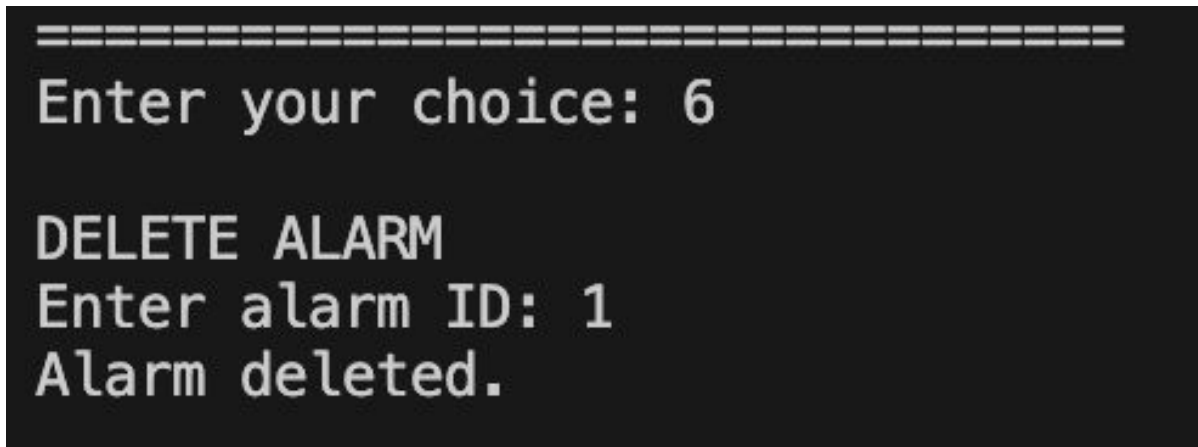
```
=====
Enter your choice: 5

EDIT ALARM
Enter alarm ID: 1
Enter new hour: 13
Enter new minute: 45
Enter new name: 00
Alarm updated.
```

Figure 6.6: Supplied console output for menu option 6.

6.7 Option 7 — Enable Alarm

Alarm ID 2 is enabled.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 30 equals signs. Below this, the text 'Enter your choice: 6' is displayed. After a blank line, the text 'DELETE ALARM' is shown. This is followed by 'Enter alarm ID: 1' and finally 'Alarm deleted.' on the last line.

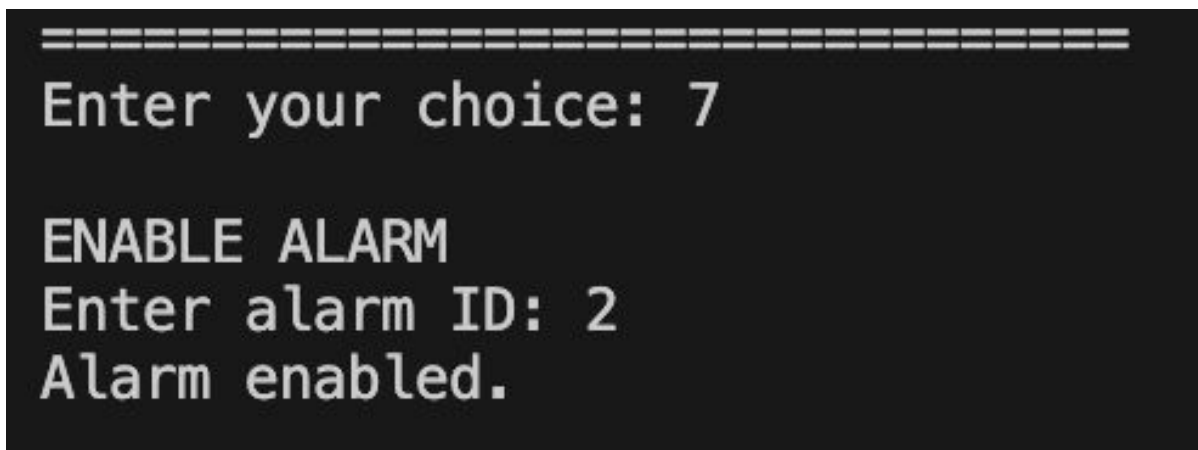
```
=====
Enter your choice: 6

DELETE ALARM
Enter alarm ID: 1
Alarm deleted.
```

Figure 6.7: Supplied console output for menu option 7.

6.8 Option 8 — Disable Alarm

Alarm ID 2 is disabled.

A terminal window with a black background and white text. At the top, there is a separator line consisting of 30 equals signs. Below this, the text 'Enter your choice: 7' is displayed. After a blank line, the text 'ENABLE ALARM' is shown. This is followed by 'Enter alarm ID: 2' and finally 'Alarm enabled.' on the last line.

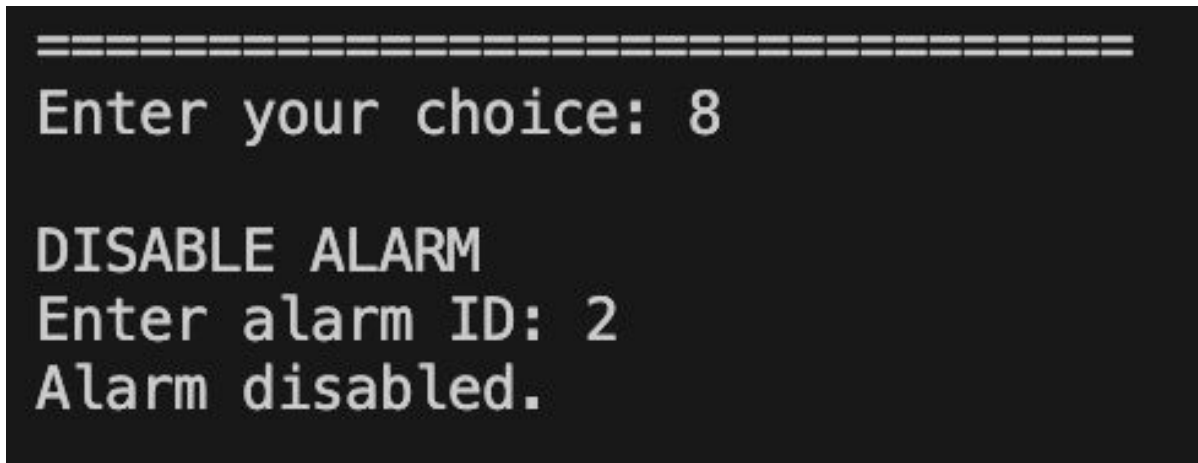
```
=====
Enter your choice: 7

ENABLE ALARM
Enter alarm ID: 2
Alarm enabled.
```

Figure 6.8: Supplied console output for menu option 8.

6.9 Option 9 — Check Alarm

The program reports that no alarm matches the current time.

A terminal window with a black background and white text. At the top, there is a separator line consisting of two rows of equals signs. Below this, the text "Enter your choice: 8" is displayed. After a few lines of blank space, the text "DISABLE ALARM" appears. This is followed by "Enter alarm ID: 2" and finally "Alarm disabled." on the last line.

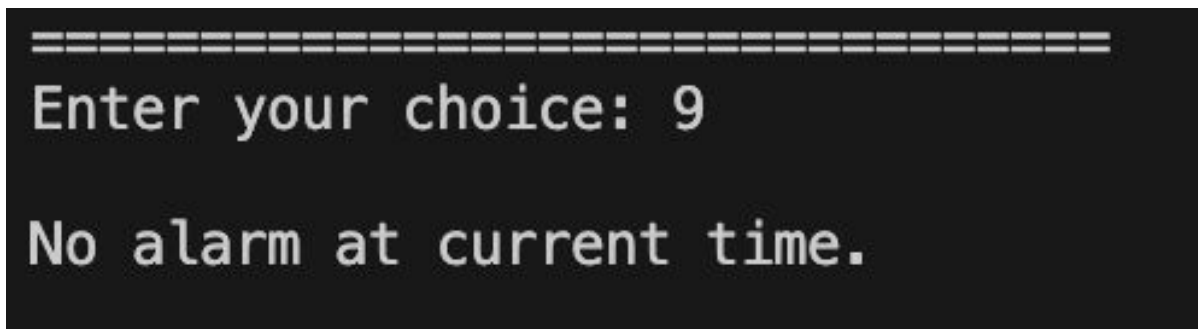
```
=====
Enter your choice: 8

DISABLE ALARM
Enter alarm ID: 2
Alarm disabled.
```

Figure 6.9: Supplied console output for menu option 9.

6.10 Option 10 — Quick Alarm

A quick alarm is created 45 minutes later, at 12:45.

A terminal window with a black background and white text. At the top, there is a separator line consisting of two rows of equals signs. Below this, the text "Enter your choice: 9" is displayed. After a few lines of blank space, the text "No alarm at current time." is displayed on the last line.

```
=====
Enter your choice: 9

No alarm at current time.
```

Figure 6.10: Supplied console output for menu option 10.

6.11 Option 11 — Search Alarm

The menu includes the alarm-search feature demonstrated by the project interface.

```
=====
Enter your choice: 10

QUICK ALARM
Alarm after how many minutes: 45
Quick alarm set at 12:45
```

Figure 6.11: Supplied console output for menu option 11.

6.12 Option 12 — Change Sound

The user selects Bell from Beep, Bell, and Buzz.

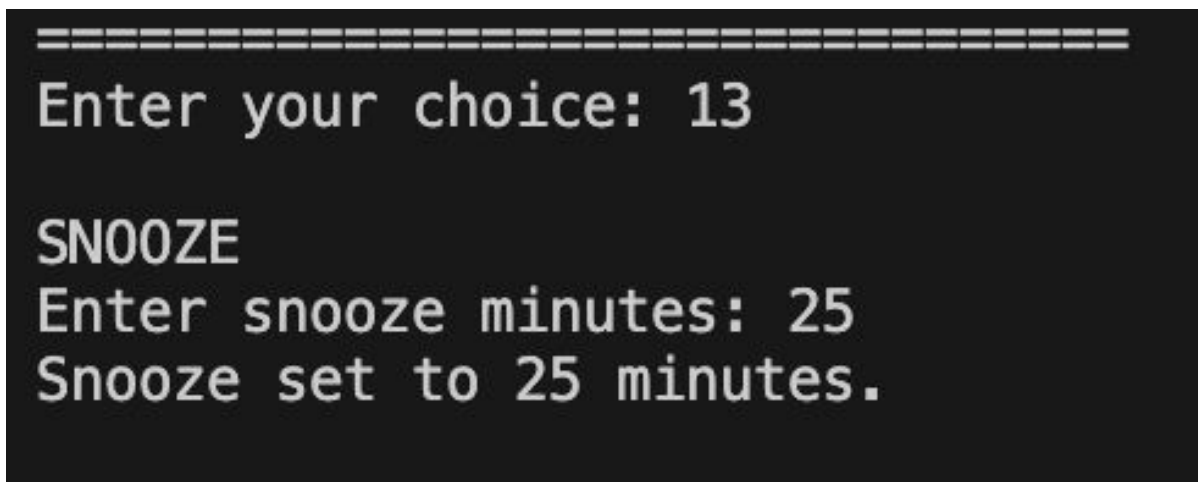
```
=====
Enter your choice: 12

CHANGE SOUND
1. Beep
2. Bell
3. Buzz
Enter choice: 2
Sound changed.
```

Figure 6.12: Supplied console output for menu option 12.

6.13 Option 13 — Set Snooze

The snooze value is set to 25 minutes.

A terminal window with a black background and white text. At the top, there is a separator line consisting of two rows of equals signs. Below this, the text 'Enter your choice: 13' is displayed. Then, the word 'SNOOZE' appears in all caps. This is followed by 'Enter snooze minutes: 25' and finally 'Snooze set to 25 minutes.'

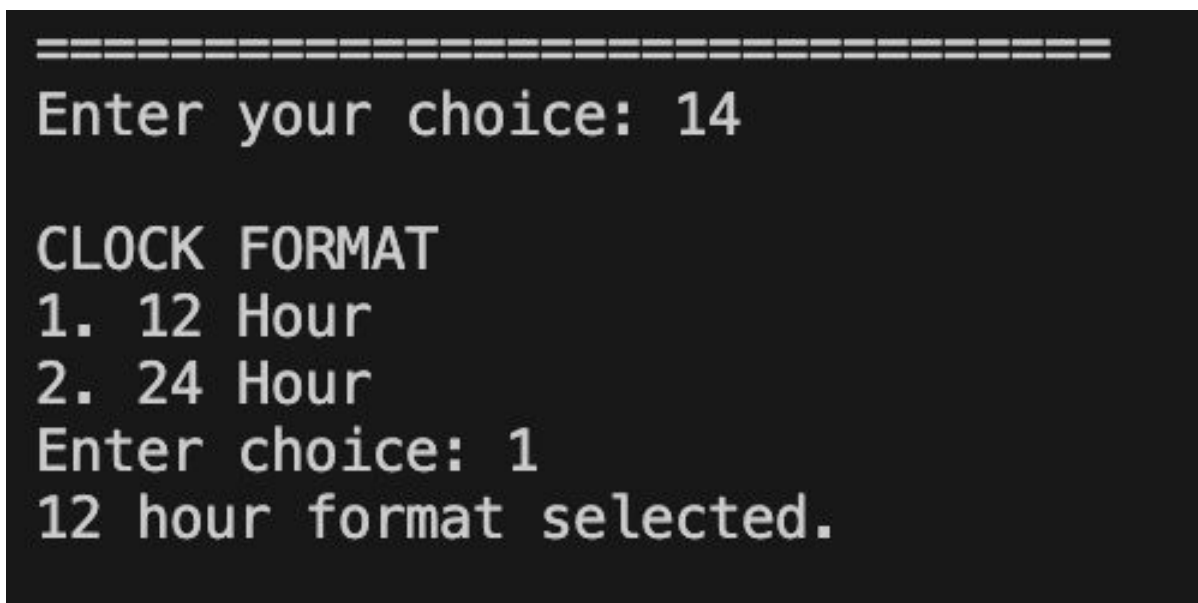
```
=====
Enter your choice: 13

SNOOZE
Enter snooze minutes: 25
Snooze set to 25 minutes.
```

Figure 6.13: Supplied console output for menu option 13.

6.14 Option 14 — Change Clock Format

The user selects the 12-hour display format.

A terminal window with a black background and white text. At the top, there is a separator line consisting of two rows of equals signs. Below this, the text 'Enter your choice: 14' is displayed. Then, the words 'CLOCK FORMAT' appear in all caps. This is followed by a numbered list: '1. 12 Hour' and '2. 24 Hour'. Then, the text 'Enter choice: 1' is displayed, and finally '12 hour format selected.'

```
=====
Enter your choice: 14

CLOCK FORMAT
1. 12 Hour
2. 24 Hour
Enter choice: 1
12 hour format selected.
```

Figure 6.14: Supplied console output for menu option 14.

6.15 Option 15 — Countdown Timer

A 55 minute 55 second countdown is started and then stopped by entering q.

```
=====
Enter your choice: 15

COUNTDOWN TIMER
Enter minutes: 55
Enter seconds: 55
Remaining: 3355 seconds
Press Enter for next second or q to stop: q
Timer stopped.
```

Figure 6.15: Supplied console output for menu option 15.

6.16 Option 16 — Stopwatch

The stopwatch is started and then stopped immediately, reporting 0 seconds.

```
=====
Enter your choice: 16

STOPWATCH
Time: 0 seconds
Press Enter to continue or q to stop: q
Stopwatch stopped.
Total time: 0 seconds
```

Figure 6.16: Supplied console output for menu option 16.

6.17 Option 17 — Statistics

The application displays total, active, disabled alarms, sound, and snooze settings.

```
=====
Enter your choice: 17

STATISTICS
Total alarms: 0
Active alarms: 0
Disabled alarms: 0
Sound: Bell
Snooze: 25 minutes
```

Figure 6.17: Supplied console output for menu option 17.

6.18 Option 18 — Delete All Alarms

The user confirms deletion and the application reports that all alarms were deleted.

```
=====
Enter your choice: 18

DELETE ALL ALARMS
Are you sure? yes/no: yes
All alarms deleted.
```

Figure 6.18: Supplied console output for menu option 18.

6.19 Option 19 — Exit

The application exits with the message “Thank you for using Digital Alarm Clock.”

```
=====
                DIGITAL ALARM CLOCK
=====
1. Set Current Time
2. Display Time
3. Add Alarm
4. View Alarms
5. Edit Alarm
6. Delete Alarm
7. Enable Alarm
8. Disable Alarm
9. Check Alarm
10. Quick Alarm
11. Enter Alarm Name
12. Change Sound
13. Set Snooze
14. Change Clock Format
15. Countdown Timer
16. Stopwatch
17. Statistics
18. Delete All Alarms
19. Exit
=====
Enter your choice: 19
Thank you for using Digital Alarm Clock.
harshvardhan@HARSHs-MacBook-Air CODE %
```

Figure 6.19: Supplied console output for menu option 19.

7. Source Code

The complete supplied Python source is included below as an appendix. The code is reproduced from the project file used for this report.

```
# =====  
# DIGITAL ALARM CLOCK  
# Python Essentials - Evaluated Course Project  
# =====  
#  
# Author: KRISHNA KUMAR SAW  
# Course: B.Tech Mechanical (AI & Robotics)  
#  
# SINGLE-FILE VERSION  
# =====  
  
current_hour = 12  
current_minute = 0  
current_second = 0  
  
alarms = []  
alarm_id = 1  
  
sound = "Beep"  
snooze = 5  
format_type = 24  
  
def set_time():  
    global current_hour, current_minute, current_second  
  
    print("\nSET CURRENT TIME")  
  
    h = int(input("Enter hour: "))
```

```

m = int(input("Enter minute: "))
s = int(input("Enter second: "))

if 0 <= h <= 23 and 0 <= m <= 59 and 0 <= s <= 59:
    current_hour = h
    current_minute = m
    current_second = s
    print("Time set successfully.")
else:
    print("Invalid time.")

```

```

def display_time():
    h = current_hour

    if format_type == 12:
        if h == 0:
            h = 12
            period = "AM"
        elif h < 12:
            period = "AM"
        elif h == 12:
            period = "PM"
        else:
            h = h - 12
            period = "PM"

        print("\nTIME:", str(h) + ":" +
              str(current_minute).zfill(2) + ":" +
              str(current_second).zfill(2), period)

    else:
        print("\nTIME:",
              str(current_hour).zfill(2) + ":" +
              str(current_minute).zfill(2) + ":" +
              str(current_second).zfill(2))

```

```

def add_alarm():
    global alarm_id

    print("\nADD ALARM")

```

```
h = int(input("Enter hour: "))
m = int(input("Enter minute: "))
name = input("Enter alarm name: ")

if 0 <= h <= 23 and 0 <= m <= 59:
    alarm = [alarm_id, h, m, name, True]
    alarms.append(alarm)
    print("Alarm added.")
    alarm_id = alarm_id + 1
else:
    print("Invalid alarm time.")
```

```
def show_alarms():
    print("\nALARM LIST")

    if len(alarms) == 0:
        print("No alarms available.")
        return

    for alarm in alarms:
        if alarm[4]:
            status = "ON"
        else:
            status = "OFF"

        print("ID:", alarm[0],
              " Time:", str(alarm[1]).zfill(2) + ":" +
              str(alarm[2]).zfill(2),
              " Name:", alarm[3],
              " Status:", status)
```

```
def find_alarm():
    number = int(input("Enter alarm ID: "))

    for alarm in alarms:
        if alarm[0] == number:
            return alarm

    return None
```

```
def edit_alarm():
    print("\nEDIT ALARM")

    alarm = find_alarm()

    if alarm == None:
        print("Alarm not found.")
        return

    h = int(input("Enter new hour: "))
    m = int(input("Enter new minute: "))
    name = input("Enter new name: ")

    if 0 <= h <= 23 and 0 <= m <= 59:
        alarm[1] = h
        alarm[2] = m
        alarm[3] = name
        print("Alarm updated.")
    else:
        print("Invalid time.")
```

```
def delete_alarm():
    print("\nDELETE ALARM")

    alarm = find_alarm()

    if alarm == None:
        print("Alarm not found.")
        return

    alarms.remove(alarm)
    print("Alarm deleted.")
```

```
def enable_alarm():
    print("\nENABLE ALARM")

    alarm = find_alarm()

    if alarm != None:
        alarm[4] = True
        print("Alarm enabled.")
```

```

else:
    print("Alarm not found.")

def disable_alarm():
    print("\nDISABLE ALARM")

    alarm = find_alarm()

    if alarm != None:
        alarm[4] = False
        print("Alarm disabled.")
    else:
        print("Alarm not found.")

def check_alarm():
    found = False

    for alarm in alarms:
        if alarm[4] and alarm[1] == current_hour:
            if alarm[2] == current_minute:
                print("\nALARM RINGING!")
                print("Alarm:", alarm[3])
                print("Sound:", sound)
                print("Beep Beep Beep!")
                found = True

    if not found:
        print("\nNo alarm at current time.")

def change_sound():
    global sound

    print("\nCHANGE SOUND")
    print("1. Beep")
    print("2. Bell")
    print("3. Buzz")

    choice = input("Enter choice: ")

    if choice == "1":

```

```
    sound = "Beep"
elif choice == "2":
    sound = "Bell"
elif choice == "3":
    sound = "Buzz"
else:
    print("Invalid choice.")
    return

print("Sound changed.")
```

```
def change_format():
    global format_type

    print("\nCLOCK FORMAT")
    print("1. 12 Hour")
    print("2. 24 Hour")

    choice = input("Enter choice: ")

    if choice == "1":
        format_type = 12
        print("12 hour format selected.")
    elif choice == "2":
        format_type = 24
        print("24 hour format selected.")
    else:
        print("Invalid choice.")
```

```
def set_snooze():
    global snooze

    print("\nSNOOZE")

    value = int(input("Enter snooze minutes: "))

    if value > 0:
        snooze = value
        print("Snooze set to", snooze, "minutes.")
    else:
        print("Invalid value.")
```

```

def quick_alarm():
    global alarm_id

    print("\nQUICK ALARM")

    minutes = int(input("Alarm after how many minutes: "))

    if minutes < 0:
        print("Invalid value.")
        return

    total = current_hour * 60 + current_minute
    total = total + minutes
    total = total % 1440

    h = total // 60
    m = total % 60

    alarm = [alarm_id, h, m, "Quick Alarm", True]
    alarms.append(alarm)

    print("Quick alarm set at",
          str(h).zfill(2) + ":" + str(m).zfill(2))

    alarm_id = alarm_id + 1


def search_alarm():
    print("\nENTER ALARM NAME")

    name = input("Enter alarm name: ")
    found = False

    for alarm in alarms:
        if name.lower() in alarm[3].lower():
            print("ID:", alarm[0],
                  "Time:", str(alarm[1]).zfill(2) + ":" +
                  str(alarm[2]).zfill(2),
                  "Name:", alarm[3])
            found = True

```

```
if not found:
    print("Alarm not found.")
```

```
def timer():
    print("\nCOUNTDOWN TIMER")

    minutes = int(input("Enter minutes: "))
    seconds = int(input("Enter seconds: "))

    if seconds < 0 or seconds > 59 or minutes < 0:
        print("Invalid time.")
        return

    total = minutes * 60 + seconds

    while total > 0:
        print("Remaining:", total, "seconds")

        choice = input("Press Enter for next second or q to stop: ")

        if choice == "q":
            print("Timer stopped.")
            return

        total = total - 1

    print("TIME UP!")
    print("Beep Beep Beep!")
```

```
def stopwatch():
    print("\nSTOPWATCH")

    seconds = 0

    while True:
        print("Time:", seconds, "seconds")

        choice = input("Press Enter to continue or q to stop: ")

        if choice == "q":
            break
```

```
seconds = seconds + 1
```

```
print("Stopwatch stopped.")  
print("Total time:", seconds, "seconds")
```

```
def statistics():
```

```
    active = 0
```

```
    inactive = 0
```

```
    for alarm in alarms:
```

```
        if alarm[4]:
```

```
            active = active + 1
```

```
        else:
```

```
            inactive = inactive + 1
```

```
    print("\nSTATISTICS")
```

```
    print("Total alarms:", len(alarms))
```

```
    print("Active alarms:", active)
```

```
    print("Disabled alarms:", inactive)
```

```
    print("Sound:", sound)
```

```
    print("Snooze:", snooze, "minutes")
```

```
def delete_all():
```

```
    print("\nDELETE ALL ALARMS")
```

```
    choice = input("Are you sure? yes/no: ")
```

```
    if choice.lower() == "yes":
```

```
        alarms.clear()
```

```
        print("All alarms deleted.")
```

```
    else:
```

```
        print("Operation cancelled.")
```

```
def menu():
```

```
    print("\n")
```

```
    print("=====")
```

```
    print("    DIGITAL ALARM CLOCK")
```

```
    print("=====")
```

```
    print("1. Set Current Time")
```

```
print("2. Display Time")
print("3. Add Alarm")
print("4. View Alarms")
print("5. Edit Alarm")
print("6. Delete Alarm")
print("7. Enable Alarm")
print("8. Disable Alarm")
print("9. Check Alarm")
print("10. Quick Alarm")
print("11. Enter Alarm Name")
print("12. Change Sound")
print("13. Set Snooze")
print("14. Change Clock Format")
print("15. Countdown Timer")
print("16. Stopwatch")
print("17. Statistics")
print("18. Delete All Alarms")
print("19. Exit")
print("=====")
```

while True:

```
    menu()
```

```
    choice = input("Enter your choice: ")
```

```
    if choice == "1":
```

```
        set_time()
```

```
    elif choice == "2":
```

```
        display_time()
```

```
    elif choice == "3":
```

```
        add_alarm()
```

```
    elif choice == "4":
```

```
        show_alarms()
```

```
    elif choice == "5":
```

```
        edit_alarm()
```

```
    elif choice == "6":
```

```
        delete_alarm()

elif choice == "7":
    enable_alarm()

elif choice == "8":
    disable_alarm()

elif choice == "9":
    check_alarm()

elif choice == "10":
    quick_alarm()

elif choice == "11":
    search_alarm()

elif choice == "12":
    change_sound()

elif choice == "13":
    set_snooze()

elif choice == "14":
    change_format()

elif choice == "15":
    timer()

elif choice == "16":
    stopwatch()

elif choice == "17":
    statistics()

elif choice == "18":
    delete_all()

elif choice == "19":
    print("Thank you for using Digital Alarm Clock.")
    break

else:
```

8. Conclusion

The **Digital Alarm Clock** project successfully demonstrates how a practical everyday concept can be implemented using fundamental Python programming techniques. The application brings together several different functions under one simple, menu-driven interface, allowing the user to interact with the program through a series of clearly defined options. Instead of focusing only on a basic clock display, the project expands the idea into a complete alarm-management application with additional time-related utilities.

The project includes **19 menu options**, covering almost every major operation required by the application. These include setting the current time, displaying the time, adding alarms, viewing alarms, editing alarms, deleting alarms, enabling and disabling alarms, checking alarms, creating quick alarms, searching alarms, changing the alarm sound, setting snooze duration, changing the clock format, using a countdown timer, using a stopwatch, viewing statistics, deleting all alarms, and exiting the program.

The execution evidence demonstrates that these different operations can be performed successfully through user input. During the demonstrated execution, the current time was set to **11:30:00**, after which the program displayed the stored time correctly. An alarm was then created, displayed in the alarm list, edited, and subsequently deleted. Another alarm was enabled and disabled to demonstrate the status-management functionality. The Check Alarm feature also successfully reported when no alarm matched the current time.

The project further demonstrated the **Quick Alarm** feature by setting an alarm 45 minutes after the current time. The program calculated the resulting time as **12:45**, showing that the application can perform basic time arithmetic. The sound setting was changed to **Bell**, while the snooze duration was configured to **25 minutes**. The clock format was also changed to the 12-hour format, demonstrating that the application can modify and maintain user-selected settings.

The **Countdown Timer** and **Stopwatch** features provide additional evidence of the program's ability to handle repeated operations using loops. The timer accepted a duration of 55 minutes and 55 seconds and was then stopped by the user. Similarly, the stopwatch was started and stopped, reporting the elapsed time. Although these functions operate as interactive simulations rather than real-time background services, they successfully demonstrate the underlying programming logic.

The **Statistics** feature provides a useful summary of the current alarm configuration by displaying the total number of alarms, active alarms, disabled alarms, selected sound, and snooze duration. The final Delete All Alarms operation demonstrated that the program can clear the stored alarm list after receiving confirmation from the user. Finally, selecting option 19 successfully terminated the application and displayed the closing message.

From an educational perspective, the project provides practical experience with several important Python concepts. The use of separate functions demonstrates **modular programming**, while the alarm list demonstrates basic data management. Conditional statements are used to validate inputs and control program behaviour, while loops keep the

menu active and allow repeated operations. String methods, Boolean values, arithmetic operations, and user input are also used throughout the application. This makes the project a useful example of how individual programming concepts can be combined to create a complete working application.

The project also has certain limitations that can guide future development. The current time is manually entered and does not automatically update according to the computer's real system clock. Alarm checking occurs when the user selects the Check Alarm option rather than automatically running in the background. The selected alarm sound is displayed as text rather than producing actual audio, and the configured snooze value is stored as a setting rather than automatically rescheduling an alarm. Alarm information is also maintained only during the current execution of the program.

These limitations do not reduce the educational value of the project. Instead, they provide clear possibilities for improvement. A future version could use the computer's real-time clock, automatically monitor alarms in the background, play actual alarm sounds, implement functional snooze behaviour, and store alarms permanently using JSON, SQLite, or another form of data storage. A graphical user interface could also make the application easier and more attractive to use.

In conclusion, the Digital Alarm Clock provides a strong example of applying basic Python programming concepts to a practical problem. It successfully combines **clock management, alarm management, search, settings, timer, stopwatch, and statistics** into one organized application. The project demonstrates not only the ability to write individual Python functions but also the ability to connect those functions into a complete menu-driven system. It therefore serves as a solid foundation for further learning and for developing more advanced applications in the future.

End of Project Report